

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester

Database Systems Engineering and Distributed Backend Development (25CS1302E)

Skilling Session 5: MongoDB – Document Database Engineering

Name	Valaboju Akshitha	Roll No.	2520080003
Course	25CS1302E	CO Mapping	CO2
Session	Skilling 5	Date	

Aim

To understand the BSON document model, collections and documents, choose between embedded and referenced document patterns, perform complete CRUD with PyMongo (Python) and Mongoose (Node.js), and query with comparison, logical, array, element and regex operators.

1. MongoDB Document Engineering with PyMongo

The Task Hub application is modeled using MongoDB documents. Users are stored in a separate collection because user information is shared by many tasks and projects. Task-specific checklist information is embedded inside the task document because it is read and updated together with the task.

Database and Collections

```
from pymongo import ASCENDING, MongoClient
client = MongoClient("mongodb://localhost:27017")
db = client["taskhub_db"]

db.drop_collection("projects")
db.drop_collection("tasks")
db.drop_collection("users")

db.users.create_index([("email", ASCENDING)], unique=True)
```

Embedded vs Referenced Patterns

Embedded pattern: checklist items and comments are stored inside the task document so they can be retrieved with the task in one read. Referenced pattern: tasks store assignee_id, while shared user data remains in the users collection.

2. CREATE Operations

```
users = db.users.insert_many([
    {"_id": 1, "name": "Chakrika", "email": "chakrika@taskhub.dev",
     "skills": ["python", "sql", "fastapi"]},
    {"_id": 2, "name": "Ananya Rao", "email": "ananya@taskhub.dev",
     "skills": ["react", "css"]},
    {"_id": 3, "name": "Rahul Verma", "email": "rahul@taskhub.dev",
     "skills": ["java", "spring", "sql"]}
])

one = db.tasks.insert_one({
    "_id": 101,
    "title": "Design ER diagram",
    "project": "Task Hub Backend",
    "status": "DONE",
    "priority": 1,
    "estimate_hours": 6,
    "assignee_id": 1,
    "tags": ["db", "design"],
    "checklist": [
        {"item": "List entities", "done": True},
        {"item": "Add cardinalities", "done": True}
    ]
})
```

The program also demonstrates insert_many() for multiple task documents and a unique email index that rejects duplicate user emails.

3. READ Operations and Query Operators

Basic Read and Projection

```

db.tasks.find_one({"_id": 101},
{"_id": 1, "title": 1, "checklist": 1})

db.tasks.find(
{"project": "Task Hub Backend"},
{"_id": 0, "title": 1, "status": 1}
)

```

Comparison Operators

\$eq, \$ne, \$gt, \$gte, \$lt, \$lte, \$in and \$nin are used to compare field values. Examples include finding tasks with estimate_hours greater than or equal to 9, or tasks whose status is TODO/IN_PROGRESS while priority is not 1.

Logical Operators

\$and, \$or, \$nor and \$not are used to combine or negate conditions. The source demonstrates priority 1 OR the ui tag, backend tasks whose status is not DONE, and tasks that are neither DONE nor priority 3.

Array Operators

\$all finds documents containing all specified array values, \$size matches an exact array length, and \$elemMatch checks for an array element satisfying multiple conditions.

Element Operators and Regex

\$exists checks whether a field exists and \$type checks its BSON type. A case-insensitive regex ^write is used to find task titles beginning with "Write".

4. Referenced Document Lookup

```

task = db.tasks.find_one({"_id": 102})
db.users.find_one(
{"_id": task["assignee_id"]},
{"_id": 0, "name": 1}
)

```

The referenced user is resolved with a second query. In the source output, task 102 resolves to Rahul Verma.

5. UPDATE Operations

```

# Update fields
db.tasks.update_one(
{"_id": 103},
{"$set": {"status": "IN_PROGRESS"},
"$inc": {"estimate_hours": 2}}
)

# Update embedded array data
db.tasks.update_one(
{"_id": 103},
{"$push": {"checklist": {"item": "GET /tasks", "done": False}},
"$addToSet": {"tags": "fastapi"}}
)

# Update multiple documents
db.tasks.update_many(
{"project": "Task Hub Frontend"},
{"$set": {"sprint": 4}}
)

# Positional update
db.tasks.update_one(
{"_id": 102, "checklist.item": "Indexes"},
{"$set": {"checklist.$done": True}}
)

# Replace and upsert
db.tasks.replace_one(
{"_id": 105},
{"title": "Write release notes v1.0",
"project": "Task Hub Frontend",
"status": "TODO", "priority": 3,
"tags": ["docs"]}
)

db.tasks.update_one(
{"_id": 999},
{"$set": {"title": "Created by upsert", "status": "TODO"}},
upsert=True
)

```

The source demonstrates \$set, \$inc, \$push, \$addToSet, positional \$, replace_one(), update_many(), and upsert. Task 103 changes to IN_PROGRESS with 14 estimated hours and gains the fastapi tag.

6. DELETE Operations

```
db.tasks.delete_one({"_id": 999})
db.tasks.delete_many({"status": "DONE"})

print(db.tasks.count_documents({}))
print(sorted(db.tasks.distinct("status")))
```

After deletion, the source output reports 4 remaining tasks and the remaining statuses are IN_PROGRESS and TODO.

7. MongoDB CRUD with Mongoose (Node.js)

The second part uses Mongoose schemas and models. User data is referenced through ObjectId, while comments are embedded as sub-documents. The task schema applies required fields, enum validation, priority limits, trimming, timestamps and a unique email constraint for users.

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true, lowercase: true }
});

const commentSchema = new mongoose.Schema(
  { author: String,
    text: { type: String, required: true },
    at: { type: Date, default: Date.now } },
  { _id: false }
);

const taskSchema = new mongoose.Schema({
  title: { type: String, required: [true, "title is required"], trim: true },
  status: { type: String,
    enum: ["TODO", "IN_PROGRESS", "DONE"], default: "TODO" },
  priority: { type: Number, min: 1, max: 3, default: 2 },
  assignee: { type: mongoose.Schema.Types.ObjectId, ref: "User" },
  tags: [String],
  comments: [commentSchema]
}, { timestamps: true });
```

8. Mongoose CRUD and Validation

Mongoose create and insertMany operations create users and tasks. Invalid task data is rejected by schema validation; the source output reports validation failures for priority, status and title.

```
const highOrMedium = await Task.find({
  priority: { $lte: 2 }
}).select("title priority -_id").sort({
  priority: 1, title: 1
}).lean();

const open = await Task.find({
  status: { $in: ["TODO", "IN_PROGRESS"] },
  tags: { $exists: true }
}).select("title status -_id").sort({ title: 1 }).lean();

const writeTasks = await Task.find({
  title: /^write/i
}).select("title -_id").lean();
```

Mongoose also demonstrates \$or for unassigned tasks, populate() for resolving a referenced user, updateOne(), updateMany(), findOneAndUpdate(), \$push for embedded comments, deleteOne() and deleteMany().

9. Key Output

Operation / Query	Observed Result
PyMongo insert_one	Inserted task ID 101
PyMongo insert_many	Inserted task IDs 102, 103, 104, 105
Unique email index	Duplicate email rejected
\$gte query	Tasks with estimate_hours >= 9 returned
\$regex query	Write FastAPI CRUD; Write release notes
Referenced lookup	Task 102 assigned to Rahul Verma
Update operations	Fields, arrays, embedded items updated
Upsert	New task created with ID 999

Delete operations	DONE task and upserted task removed
Mongoose validation	Invalid priority/status/title rejected
Mongoose populate	Build dashboard page resolved to Ananya Rao
Mongoose final tasks	Build dashboard page; Write FastAPI CRUD

Result

Task documents embedded their checklist and comments so they could be read together with the task, while users were referenced through assignee_id and resolved with a second query in PyMongo and populate() in Mongoose. Complete CRUD operations were demonstrated, including insert_one/insert_many, find with projection, update_one/update_many with \$set, \$inc, \$push, \$addToSet and positional \$, replace_one, upsert, delete_one and delete_many. Comparison, logical, array, element and regex operator families returned the expected documents, while the unique index and Mongoose schema validation rejected invalid data.